

Chapitre 4

Ordonnancement de tâches dépendantes

1. Prise en compte des contraintes de précédence

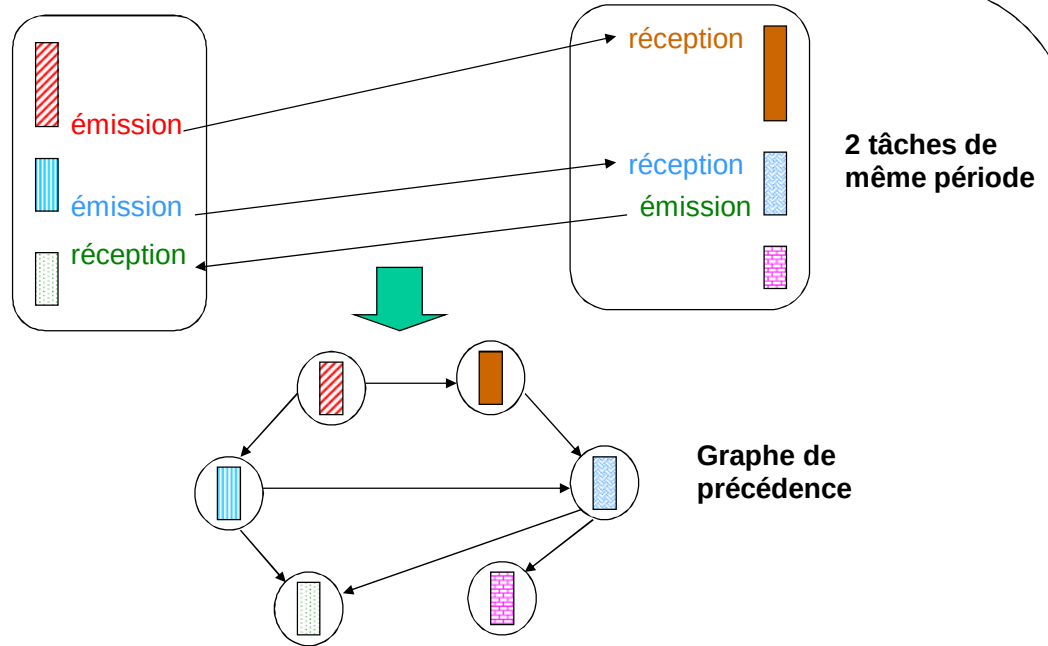
→ Prise en compte de la communication

→ Synchronisation

● **Principe** Se ramener à l'ordonnancement de tâches indépendantes

● **Méthode** Ajustement des dates de réveil et échéances
Utilisation d'un algorithme classique : RM, DM ou EDF
Égalité des priorités $\Rightarrow$ utilisation du graphe de précédence

$$T_1 \longrightarrow T_2 \Rightarrow Prio_1 > Prio_2$$



● **Résultat**

Le système ajusté est ordonnançable par l'algorithme choisi



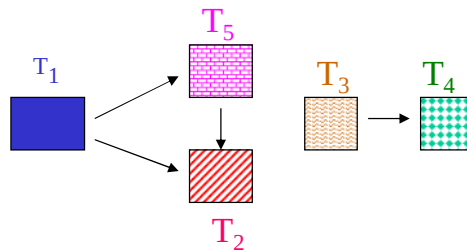
Le système initial est ordonnançable

Précédence et RM

Règles d'ajustement : $r_i^* = \text{Max}\{r_i, r_k^*, T_k \rightarrow T_i\}$

$\text{Prio}_i^* = \text{Max}\{\text{Pro}_i, \text{Prio}_k^* + a, T_i \rightarrow T_k\}, \quad a > 0$

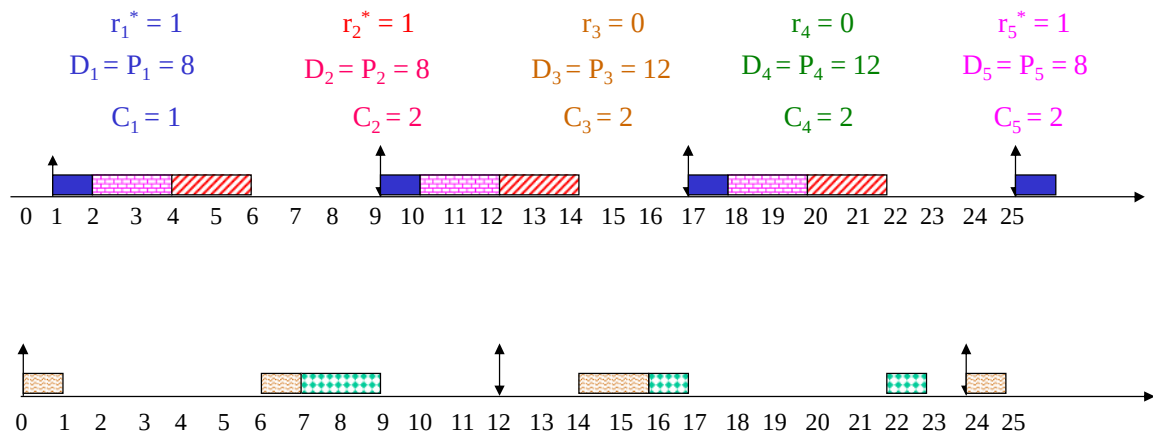
$r_1 = 1$	$r_2 = 0$	$r_3 = 0$	$r_4 = 0$	$r_5 = 0$
$D_1 = P_1 = 8$	$D_2 = P_2 = 8$	$D_3 = P_3 = 12$	$D_4 = P_4 = 12$	$D_5 = P_5 = 8$
$C_1 = 1$	$C_2 = 2$	$C_3 = 2$	$C_4 = 2$	$C_5 = 2$



$r_1^* = 1$	$r_2^* = 1$	$r_3^* = 0$	$r_4^* = 0$	$r_5^* = 1$
-------------	-------------	-------------	-------------	-------------

Prio₄ < Prio₃ < Prio₂ < Prio₅ < Prio₁

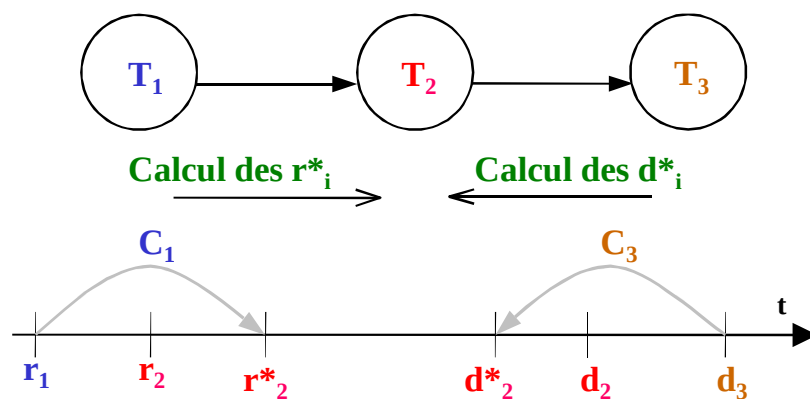
Exemple RM et précedence



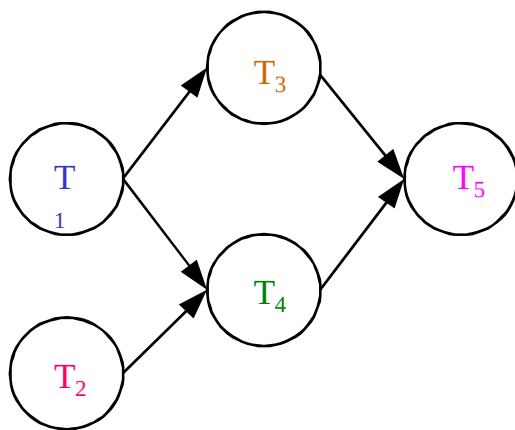
$\text{Prio}_4 < \text{Prio}_3 < \text{Prio}_2 < \text{Prio}_5 < \text{Prio}_1$

Précédence et EDF

Règle d'ajustement : $r_i^* = \max_k \{r_i, r_k^* + C_k, T_k \rightarrow T_i\}$
 $d_i^* = \min_k \{d_i, d_k^* - C_k, T_i \rightarrow T_k\}$

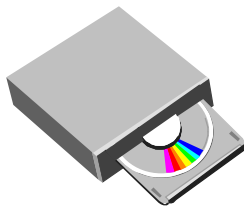


Exemple EDF et précédence



Tâche	Paramètres des tâches			Paramètres transformés	
	r_i	C_i	d_i	r^*_i	d^*_i
T₁	0	1	5	0	3
T₂	5	2	7	5	7
T₃	0	2	5	1	5
T₄	0	1	10	7	9
T₅	0	3	12	8	12

2. Prise en compte des contraintes de partage de ressources



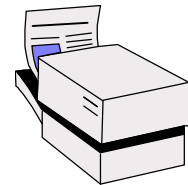
Fichiers, BD



E/S

Utilisation de **ressources critiques**

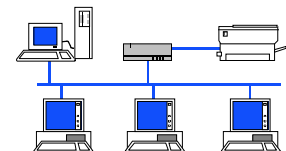
- Mono ou multi exemplaires
- Mode lecture/écriture
- Demande de 1 ou n ressources



Ressource physique

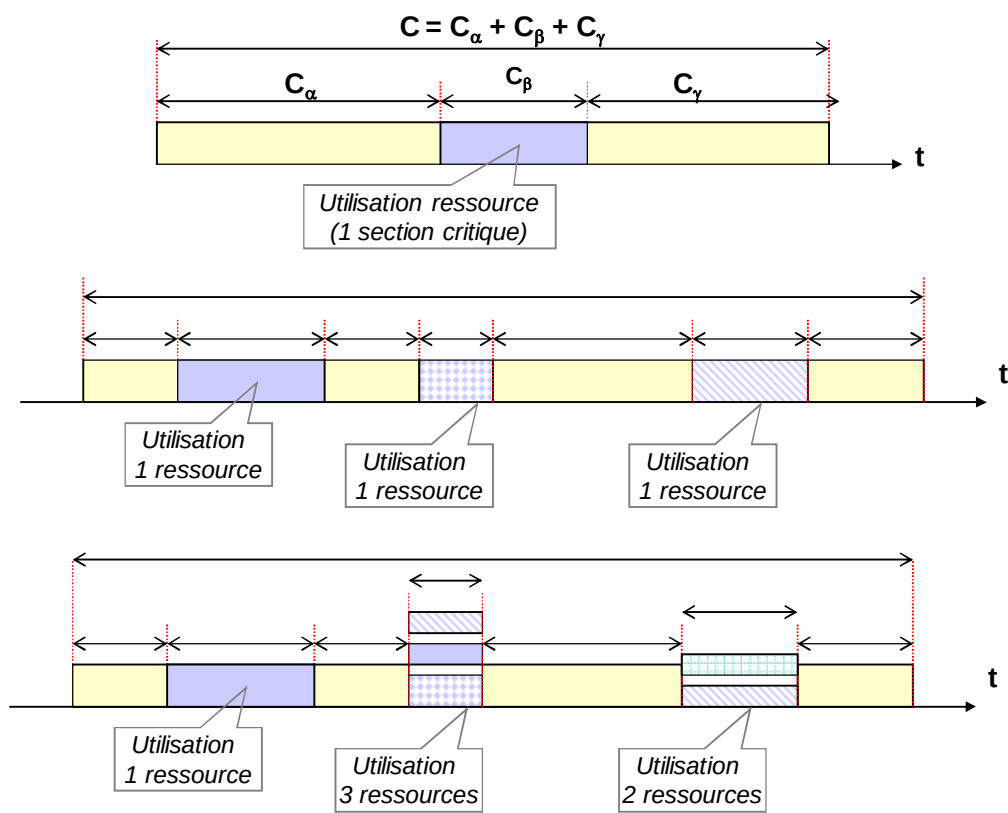
Difficultés :

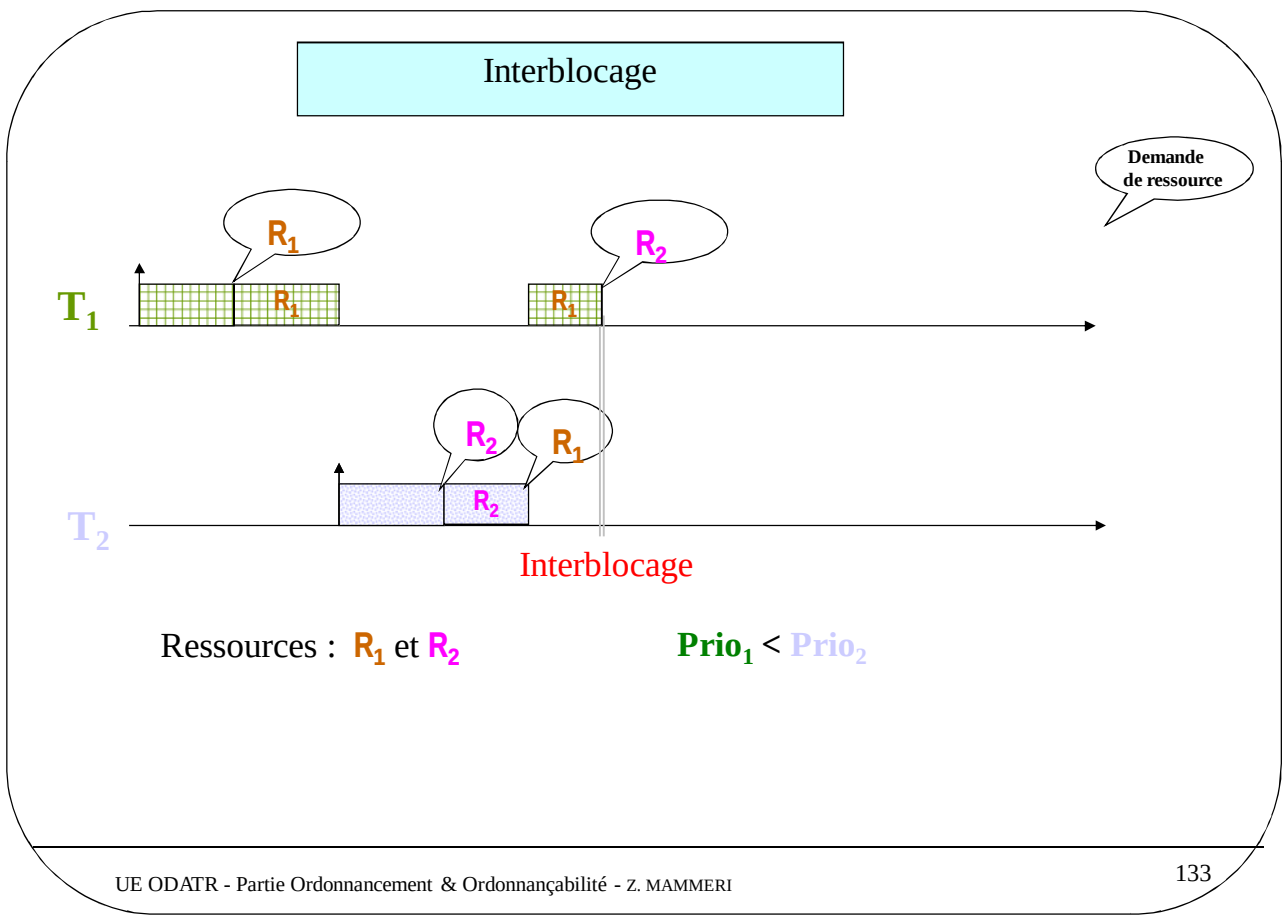
- Interblocages
- Inversion de priorités
- Instabilité des algorithmes

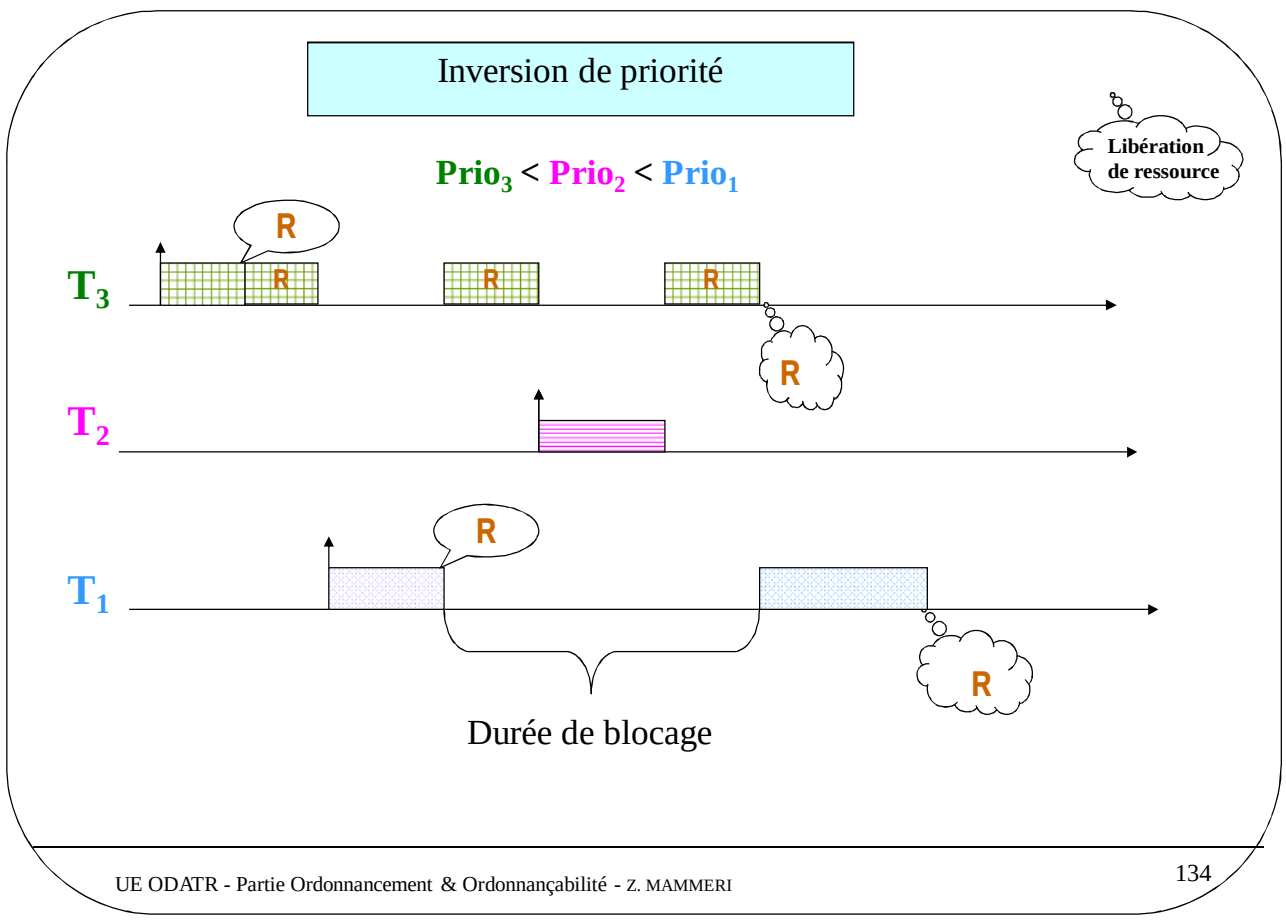


Réseau

Ordonnancement : NP-difficile, pas d'algo optimal (Mok)







Instabilité de EDF (1/2)

$$r_1 = r_2 = r_3 = 0$$

$$D_1 = P_1 = 16$$

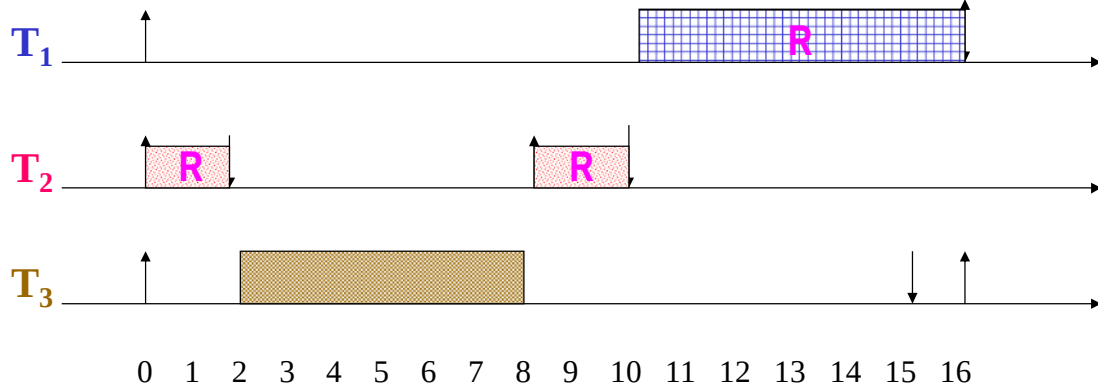
$$D_2 = 2 \quad P_2 = 8$$

$$D_3 = 15 \quad P_3 = 16$$

$$C_1 = 6$$

$$C_2 = 2$$

$$C_3 = 6$$



Instabilité de EDF (2/2)

$$r_1 = r_2 = r_3 = 0$$

$$D_1 = P_1 = 16$$

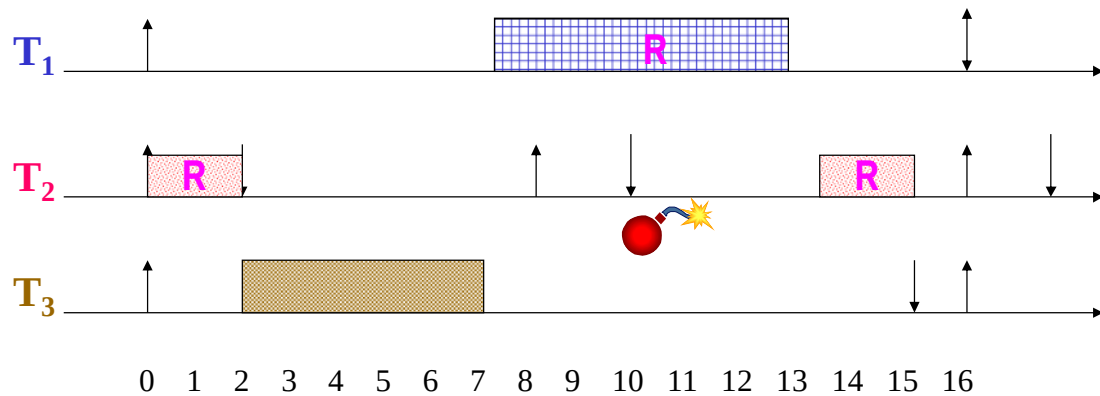
$$C_1 = 6$$

$$D_2 = 2 \quad P_2 = 8$$

$$C_2 = 2$$

$$D_3 = 15 \quad P_3 = 16$$

$$C_3 = 5$$



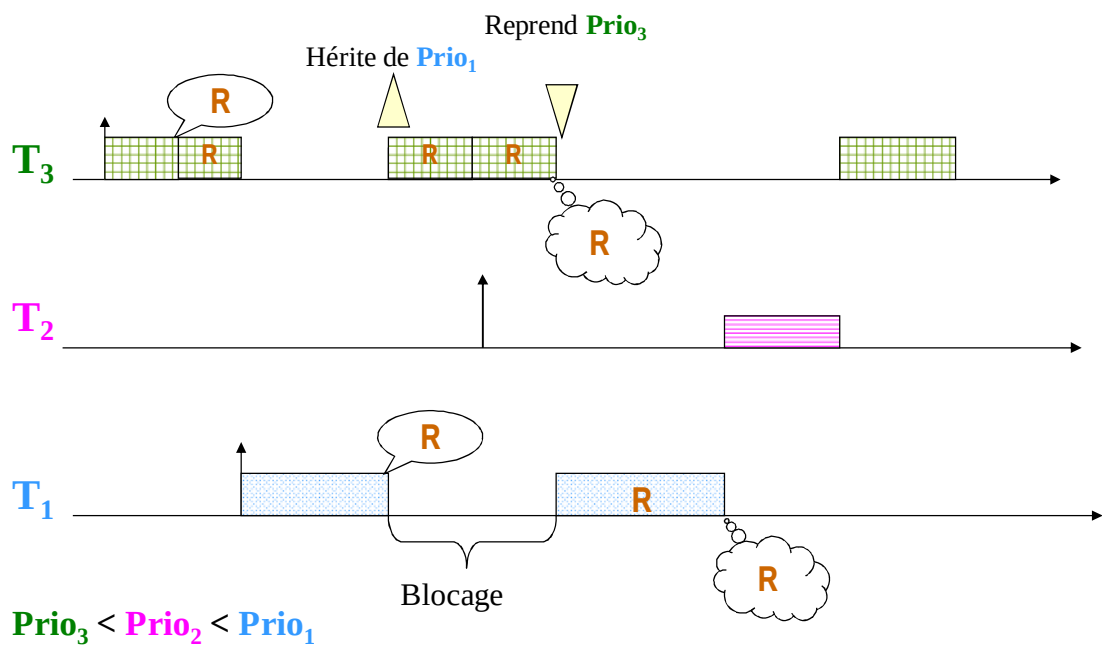
Protocole à priorité héritée : PIP (Priority Inheritance Protocol) (Sha, Rajkumar et Lehoczky)

- **Principe**

- La tâche T_i qui possède une ressource critique prend une priorité égale au $\text{Max}\{\text{Prio}_i, \text{Prio}_k \mid T_k \text{ en attente de la section critique}\}$.
- Quand T_i sort de la SC, elle reprend sa **priorité initiale**.

- **Utilisable pour les ressources mono exemplaires**

Protocole à priorité héritée PIP



Propriétés de PIP

- **Durée de blocage maxi** de n'importe quelle tâche ordonnancée avec PIP
 $\leq \text{Min}(\text{nb_tâches}, \text{nb_ressources}) * \text{pire_durée_SC}$

- **Pour une tâche i
et une seule ressource R :**

$$B_i = \max_{\forall k \notin hp(i)} (SC_k(R)) + \sum_R \max_{\forall j \in hp(i)} (durée_SC_j(R))$$

$hp(i)$: ensemble de tâches plus prioritaires que τ_i . R : toute ressource utilisée par une tâche $\in hp(i)$.

- **Condition suffisante pour RM**

- **Condition suffisante pour EDF**

$$\max_{i=1, \dots, n} \left(\frac{B_i}{P_i} \right) + \sum_{k=1}^n \frac{C_k}{P_k} \leq n(2^{1/n} - 1) \quad \forall i = 1, 2, \dots, n \Rightarrow \left(\frac{C_i + B_i}{D_i} \right) + \sum_{k=1}^{k=i-1} \frac{C_k}{D_k} \leq 1$$

B_i : durée maxi de blocage de la tâche T_i en attente de SC.

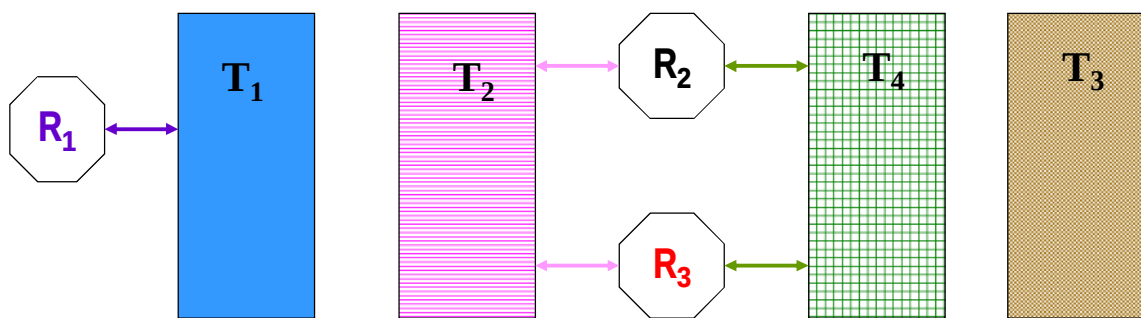
- **PIP ne gère pas les interblocages**

Protocole à priorité plafond : PCP (Priority Ceiling Protocol) (Sha, Rajkumar et Lehoczky)

● Principe

- Chaque ressource R_x possède une **priorité plafond** égale à $\text{Max}\{\text{Prio}_k \mid T_k \text{ pouvant accéder à } R_x\}$.
- On note Π_s le maximum des priorités plafonds des ressources en cours d'utilisation par des tâches. Π_s est égal à une valeur inférieure à la priorité la plus basse quand aucune ressource n'est utilisée.
- Quand une tâche T_i veut accéder à une ressource R_x qui est libre :
 - Cas 1 : si Prio_i est supérieur à Π_s , alors T_i obtient R_x et Π_s est mis à jour.
 - Cas 2 : si Prio_i n'est pas supérieur à Π_s , alors T_i obtient R_x seulement si T_i est la tâche qui détient la (ou les) ressource(s) dont le plafond est égal à Π_s .
 - Autres cas : T_i est bloquée.
- Une tâche T_i qui est en SC prend la priorité $\text{Prio} = \text{Max}\{\text{Prio}_j, \text{Prio}_k \mid T_k \text{ en attente de SC}\}$. En d'autres termes, si une tâche T_i bloque une tâche T_j , T_i hérite de Prio_j si $\text{Prio}_j > \text{Prio}_i$. Quand T_i sort de la SC, elle reprend la priorité quelle avait avant d'entrer en SC.

Exemple de fonctionnement de PCP (1/4)



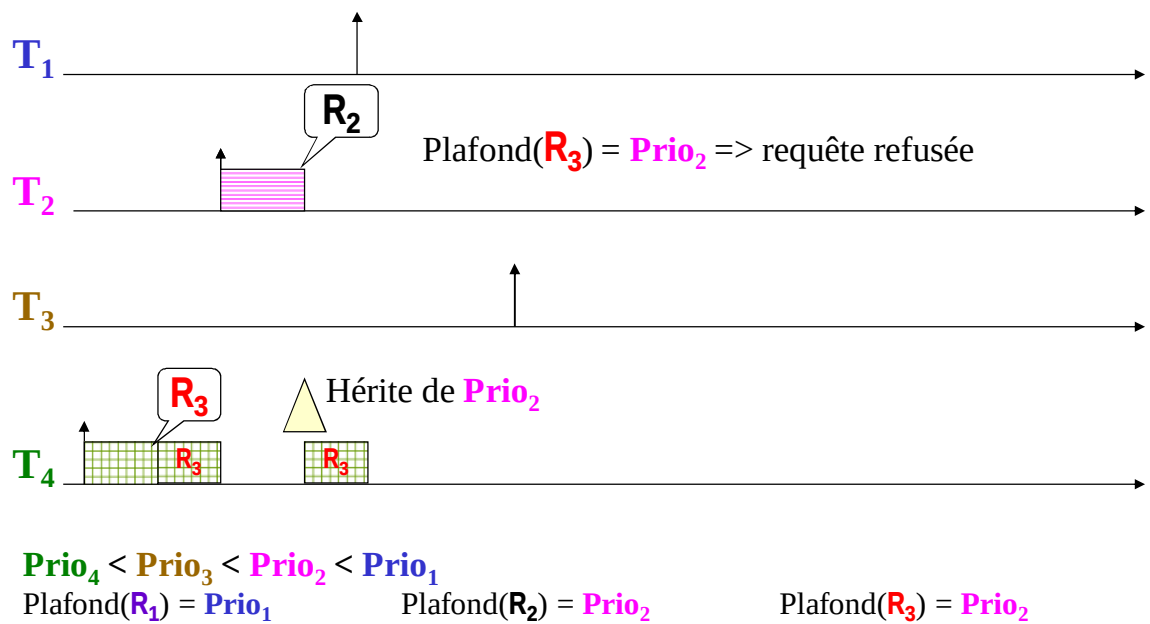
$\text{Prio}_4 < \text{Prio}_3 < \text{Prio}_2 < \text{Prio}_1$

Plafond(R_1) = Prio_1

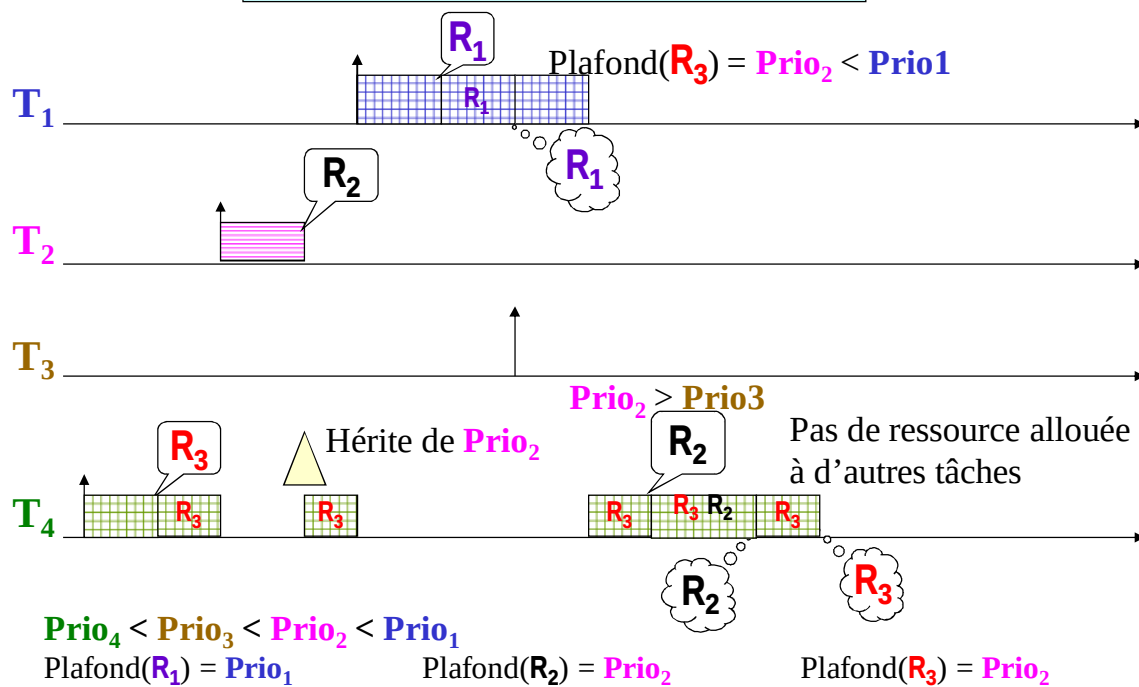
Plafond(R_2) = Prio_2

Plafond(R_3) = Prio_2

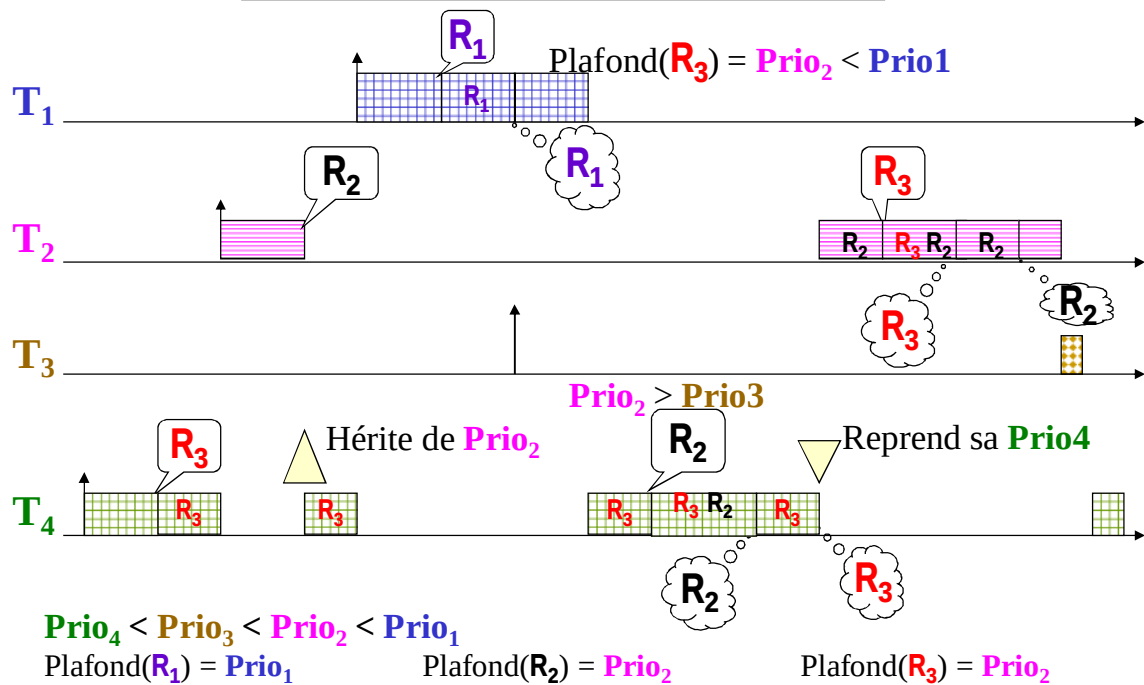
Exemple de fonctionnement de PCP (2/4)



Exemple de fonctionnement de PCP (3/4)



Exemple de fonctionnement de PCP (4/4)



Propriétés de PCP

- Utilisable pour les ressources mono exemplaires

- **Durée de blocage de la tâche la plus prioritaire $\leq$ pire_durée_SC**

Avec PCP, la tâche avec la priorité la plus élevée peut être bloquée le temps d'une SC au plus

- **Pour une tâche i :** $B_i = \max_{\forall R, \forall j \in hp(i)} (durée_SC_j(R))$

$hp(i)$: ensemble de tâches plus prioritaires que τ_i . R : toute ressource utilisée par une tâche $\in hp(i)$.

- **Condition suffisante pour EDF et tâches périodiques** (Chen et Lin)

$$\sum_{i=1}^n \frac{C_i + B_i}{P_i} \leq 1$$

B_i : temps maxi de blocage de la tâche T_i en attente de SC.

Dans la CS, les tâches sont ordonnées selon l'ordre croissant des D_i

- **PCP évite les interblocages**

Protocole à priorité à pile : SRP (Stack Resource Policy) - (Backer)

● Principe

- Chaque tâche T_i est caractérisée par une priorité $Prio_i$ (statique ou dynamique) et un **niveau de préemption** π_i (statique).
 π_i se calcule à partir de paramètres tels que D_i , C_i , **Importance_i** ...
 T_i ne peut préempter T_k que si $\pi_i > \pi_k$
- Chaque ressource R_x existe en NR_x exemplaires ($NR_x \geq 1$).
 R_x a un plafond courant C_x égal au niveau de préemption le plus élevé des tâches qui pourraient être bloquées en attente de R_x quand il y a nr_x exemplaires disponibles de R_x :

$$C_x(nr_x) = \text{Max} \left[\{0\} \cup \{\pi_k \mid nr_x < \mu_x(T_k) \} \right]$$

nr_x : Nombre d'exemplaires de R_x actuellement disponibles

$\mu_x(T_k)$: Nombre maxi d'exemplaires de R_x que peut demander la tâche T_k

→ Le plafond (Π_s) d'un système avec m ressources partagées est égal au plafond courant le plus élevé des ressources :

$$\Pi_s = \text{Max}[C_x \text{ } x=1, \dots, m]$$

→ Π_s est initialisé à 0 et modifié à chaque acquisition/libération de ressource.

→ **Test de préemption** : Si la condition « T_i est la plus prioritaire des tâches prêtes et $\pi_i > \Pi_s$ » est vraie T_i est exécutée, sinon elle reste dans la file des tâches prêtes. Cette condition est testée à chaque fois que Π_s décroît (c-à-d, quand une ressource est libérée).

● Conséquence du principe précédent

→ Avec le protocole SRP, une tâche peut être bloquée dès son démarrage et non au moment où elle demande une ressource.

→ Une tâche peut être bloquée par le test de préemption même si elle n'utilise aucune ressource, ce qui permet d'éviter l'inversion de priorité.

→ Le passage du test de préemption par une tâche T_i signifie que les ressources actuellement disponibles sont suffisantes pour satisfaire les besoins de T_i et de toutes les tâches qui pourraient la bloquer. Cela signifie aussi que toute tâche démarrée ne sera plus bloquée à cause d'une ressource. Ainsi, les situations d'interblocage sont évitées.

Exemple SRP (1/5)

3 exemplaires de $\mathbf{R}_1$ 1 exemplaires de $\mathbf{R}_2$ 3 exemplaires de $\mathbf{R}_3$

$\mathbf{T}_1$:
demande(1, $\mathbf{R}_3$)
...
demande(1, $\mathbf{R}_1$)
...
libère (1, $\mathbf{R}_1$)
...
libère(1, $\mathbf{R}_3$)

$r_1=2, D_1=9, \pi_1=3$

$\mathbf{T}_2$:
demande(3, $\mathbf{R}_3$)
...
demande(1, $\mathbf{R}_2$)
...
libère (1, $\mathbf{R}_2$)
...
libère (3, $\mathbf{R}_3$)
...
demande(2, $\mathbf{R}_1$)
...
libère(2, $\mathbf{R}_1$)

$r_2=1, D_2=16, \pi_2=2$

$\mathbf{T}_3$:
demande(1, $\mathbf{R}_2$)
...
demande(3, $\mathbf{R}_1$)
...
libère (3, $\mathbf{R}_1$)
...
libère(1, $\mathbf{R}_2$)
...
demande(1, $\mathbf{R}_3$)
...
libère(1, $\mathbf{R}_3$)

$r_3=0, D_3=20, \pi_3=1$

Exemple SRP (2/5)

Paramètres de tâches et demandes en ressources

	D_i	π_i	μ_1	μ_2	μ_3
T_1	9	3	1	0	1
T_2	16	2	2	1	3
T_3	20	1	3	1	1

Plafonds dynamiques des ressources (en fonction du nombre d'exemplaires disponibles)

- : cas impossible

	$C_R(3)$	$C_R(2)$	$C_R(1)$	$C_R(0)$
R_1	0	1	2	3
R_2	-	-	0	2
R_3	0	2	2	3

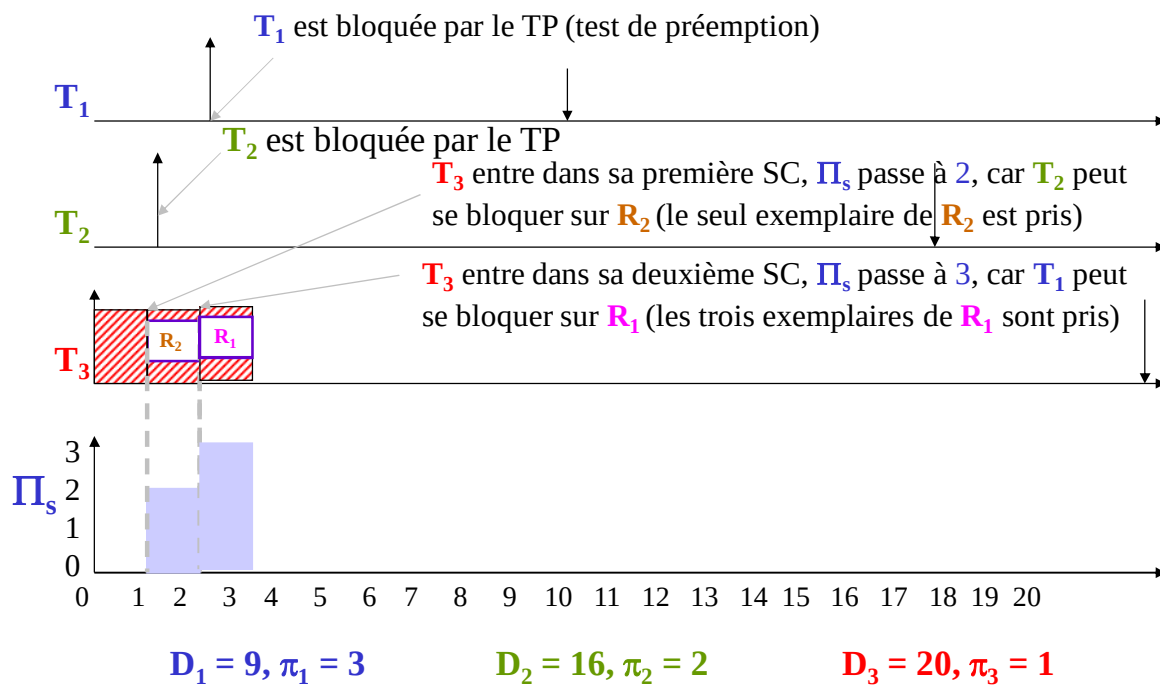
Lorsque 2 exemplaires de R_1 sont libres, la seule tâche qui pourrait être bloquée est T_3 , car elle exige 3 exemplaires de R_1 . Par conséquent $C_1(2) = \text{Max}[0, \pi_3] = 1$.

Lorsque 1 exemplaire de R_1 est libre, les tâches qui pourraient être bloquées sont T_2 et T_3 , car elles exigent chacune plus d'un exemplaire de R_1 . Par conséquent $C_1(1) = \text{Max}[0, \pi_2, \pi_3] = 2$.

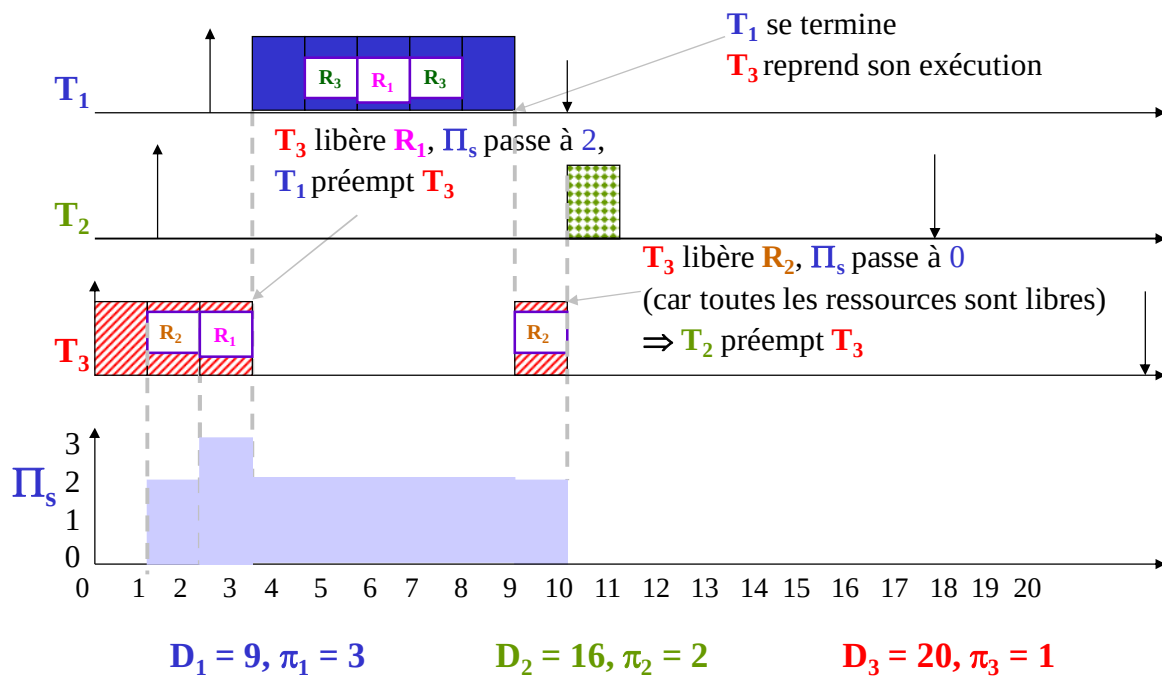
Lorsque aucun exemplaire de R_1 n'est libre, toutes les trois tâches pourraient être bloquées. Par conséquent $C_1(0) = \text{Max}[0, \pi_1, \pi_2, \pi_3] = 3$.

C'est un tableau statique utilisé par le Test de préemption

Exemple SRP + EDF (3/5)



Exemple SRP + EDF (4/5)



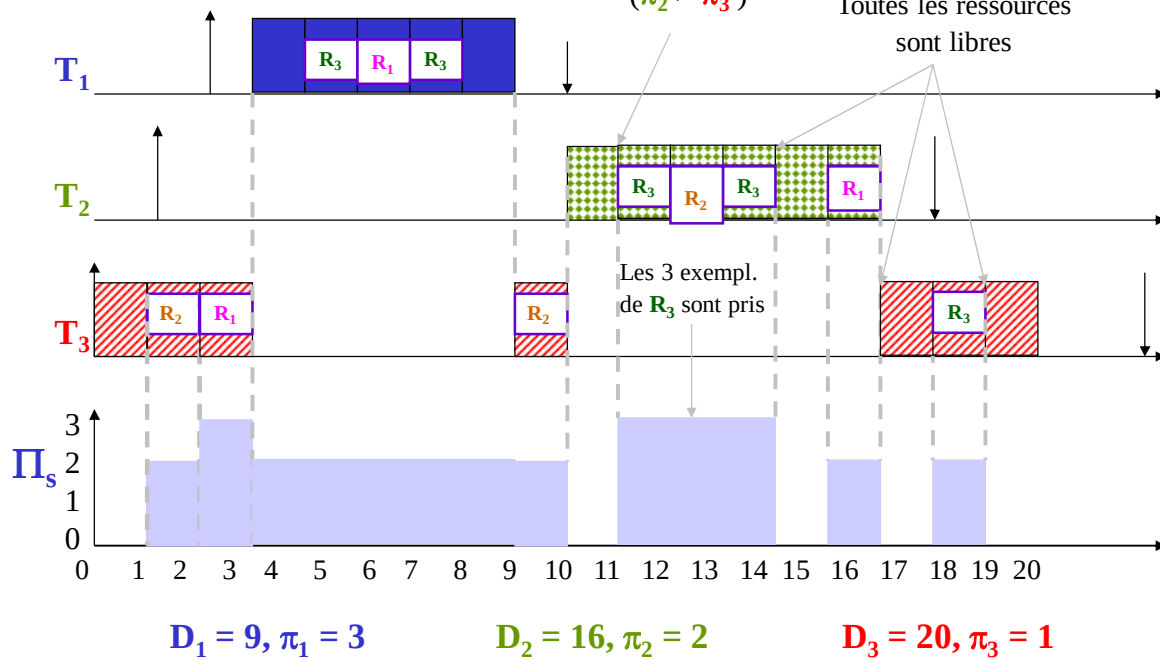
Exemple SRP + EDF (5/5)

Toutes les ressources sont libres

T_2 n'est plus interrompue

$(\pi_2 > \pi_3)$

Toutes les ressources
sont libres



Propriétés de SRP

- Utilisable pour les ressources multi exemplaires
- Utilisable avec EDF
- Durée de blocage $\leq$ pire_durée_SC
- Condition suffisante pour EDF et tâches périodiques (Baker)

$$\forall k = 1, 2, \dots, n \Rightarrow \left(\sum_{i=1}^{i=k} \frac{C_i}{D_i} \right) + \frac{B_k}{D_k} \leq 1$$

B_i : temps maxi de blocage de la tâche T_i en attente de SC.

Dans la CS, les tâches sont ordonnées selon l'ordre croissant des D_i

- Gère les interblocages
- Minimise les changements de contexte

Comparaison des principaux protocoles de gestion de ressources

	Déclaration préalable des ressources	Prévention d'interblocage	Nombre de blocages	Calcul du de temps de blocage	Moment de blocage
Protocole à héritage de priorité (Priority Inheritance Protocol)	Non (transparent)	Non	$\text{Min}(n,m)$	Difficile	Au Moment de l'accès
Protocole à priorité plafond (Priority Ceiling Protocol)	Oui	Oui	1	Facile	Au Moment de l'accès
Protocole à pile de priorité (Stack Resource Policy)	Oui	Oui	1	Facile	Au moment de la préemption

n : nombre de tâches

m : nombre de ressources

Quelques résultats [stankovic 95]

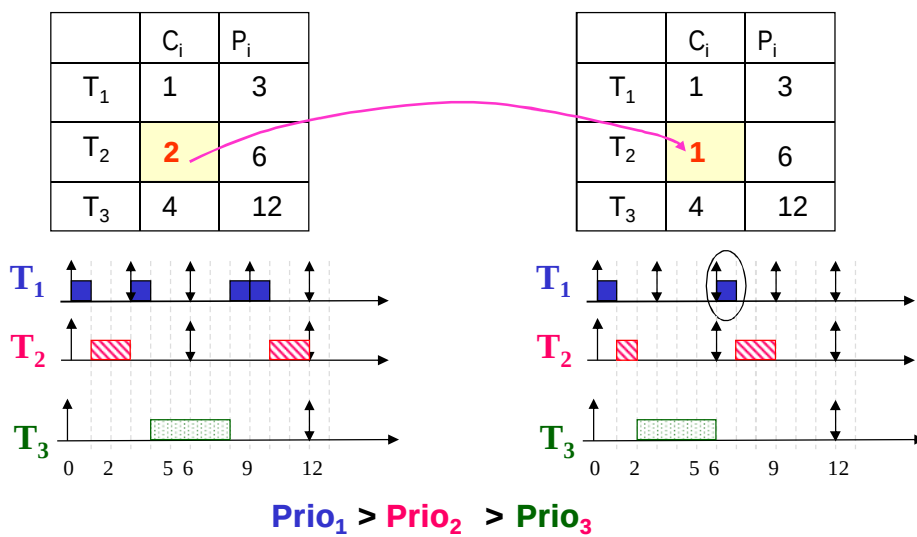
- En présence de contraintes d'exclusion mutuelle, il est impossible de trouver un ordonnanceur en ligne optimal.
- Le problème de décider s'il est possible d'ordonnancer des tâches périodiques utilisant les sémaphores pour gérer l'exclusion mutuelle est NP-difficile.
- Le problème d'ordonnancement, avec 2 processeurs, de tâches indépendantes, sans ressources, avec des temps d'exécution quelconques est NP-complet.
- EDF n'est pas optimal dans le cas multiprocesseur.
- Dans le cas de 2 processeurs ou plus, aucun ordonnancement fondé sur les délais (comme EDF par exemple) ne peut être optimal sans connaissances a priori des délais, des temps d'exécution et des instants de réveil.

La plupart des problèmes d'ordonnancement $\in$ NP

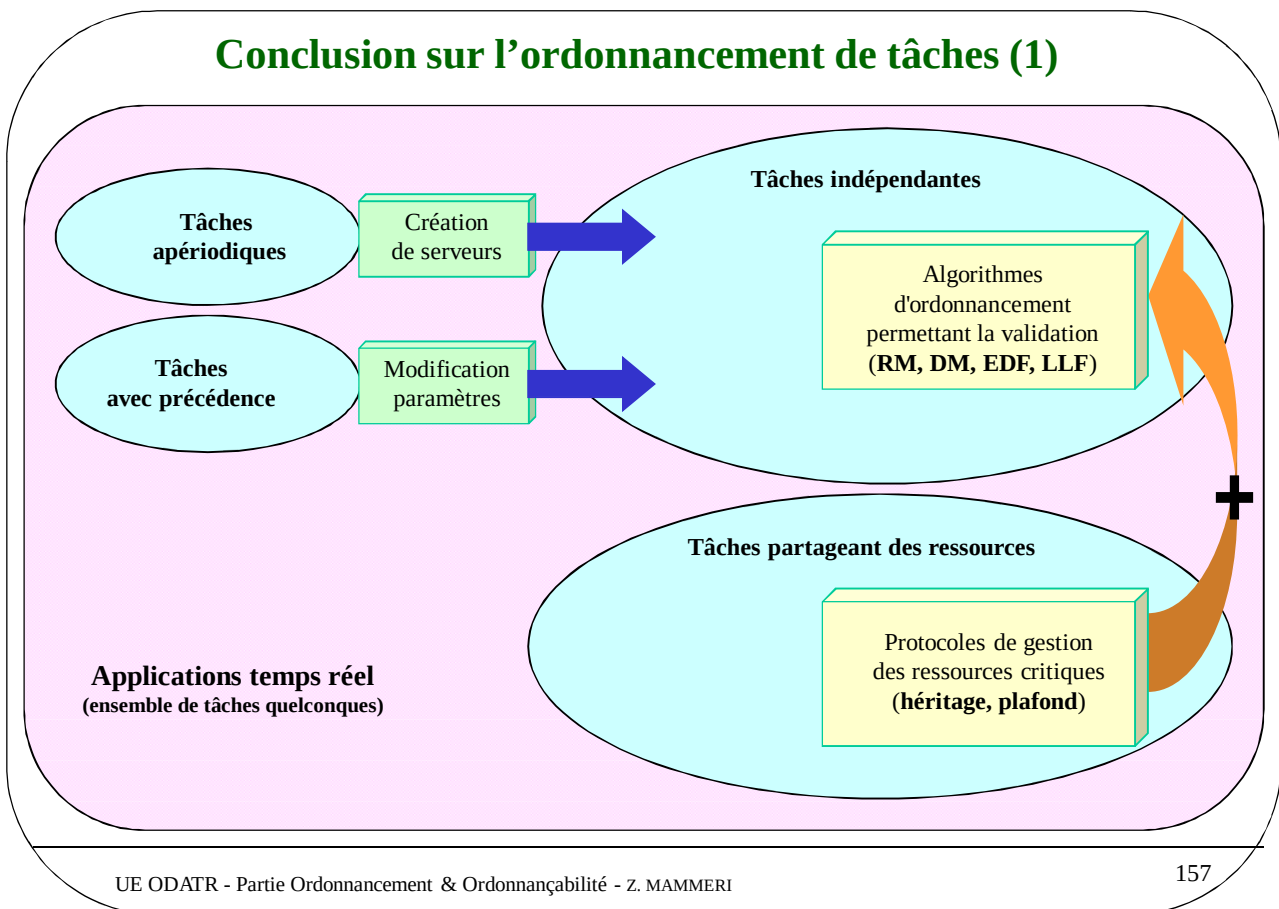
$\Rightarrow$ Heuristiques

Anomalies de l'ordonnancement

- Ne pas se fier à ce qui semble 'naturel'/'évident'.
- De nombreux types d'anomalies sont publiés.
- Exemple : cas de l'ordonnancement non préemptif à priorité fixe



Conclusion sur l'ordonnancement de tâches (1)



Conclusion sur l'ordonnancement de tâches (2)

- **Détermination efficace des paramètres de tâches (Cmax, Périodes, ...)**
- **Recherche de conditions d'ordonnançabilité**
(difficultés pour les apériodiques, le partage de ressources, la répartition)
- **Meilleure expérimentation des algorithmes**
(connaissance plus précise de leurs performances)
- **Choix efficace d'un algorithme en fonction des contraintes de tâches**
- **Ordonnancement temps réel et multimédia**
- **Ordonnancement temps réel et réseaux, Internet**
- **Application des techniques approchées (algo génétiques, réseaux de neurones, logique floue ...)**